

# Tarea Corta 1

## Seguridad del software

Kener Adiel Castillo

2023179846

13 de mayo 2026

### Fase 1: Reconocimiento y desensamblado

(1) Ejecute el binario e intente ingresar al menos tres códigos de prueba diferentes (uno corto, uno largo y uno de la longitud que el programa espera). Documente con capturas de pantalla los mensajes que obtiene en cada caso. ¿Qué puede inferir sobre las validaciones que realiza el programa solo a partir de estos mensajes?

```
~/Music > ./software_premium_04
bash: ./software_premium_04: Permission denied

~/Music > chmod u+x software_premium_04

~/Music > ./software_premium_04
=====
SOFTWARE PREMIUM v3.2.4
Ingrese su codigo de licencia
=====
Codigo (8 caracteres): al23
[-] Licencia invalida. Verifique su codigo.
[-] Contacte soporte: soporte@softwarepremium.co.cr

~/Music > ./software_premium_04
=====
SOFTWARE PREMIUM v3.2.4
Ingrese su codigo de licencia
=====
Codigo (8 caracteres): abc1345678
[-] Licencia invalida. Verifique su codigo.
[-] Contacte soporte: soporte@softwarepremium.co.cr

~/Music > ./software_premium_04
=====
SOFTWARE PREMIUM v3.2.4
Ingrese su codigo de licencia
=====
Codigo (8 caracteres): al234567
[-] Licencia invalida. Verifique su codigo.
[-] Contacte soporte: soporte@softwarepremium.co.cr

~/Music > █
```

Al ejecutar el programa se probaron tres tamaños de string diferentes: 3 (corto), 10 (largo) y 8 caracteres (el esperado).

En las tres pruebas da el mismo mensaje:

“[-] Licencia invalida. Verifique su codigo

[-] Contacte soporte: [soporte@softwarepremium.co.cr](mailto:soporte@softwarepremium.co.cr)”

Por lo que se puede inferir que no se valida el tamaño.

**(2)** Ejecute `strings software_premium` y analice la salida. ¿Qué cadenas relevantes identifica? ¿Qué nombres de funciones o constantes aparecen? ¿Qué le sugieren estos nombres sobre el funcionamiento interno del programa?

Cadenas relevantes

Funciones:

fgets  
stdin  
puts  
strcspn  
verificar\_licencia  
MULTIPLICADOR  
MODULO\_BASE  
mostrar\_flag

Constantes:

/lib64/[ld-linux-x86-64.so.2](#) (ruta)  
H=`@@` (possible formato)  
[\*] %s  
9\*3\$"  
<j?o;h>j'  
[+] Licencia valida! Funciones premium activadas. (mensaje de exito)

En el código parece haber una ruta a un programa, se identifica diferentes funciones de C y algunas propias del programa. Hay algunas constantes extrañas que parecieran ser algún tipo de formato.

**(3)** Genere el desensamblado con `objdump -d -M intel software_premium > desensamblado.txt`. Localice la función principal de validación en el archivo generado. ¿Cómo la identificó? Incluya una captura de pantalla del segmento de código desensamblado de dicha función.

```

175
176 0000000000401208 <verificar_licencia>:
177 401208: f3 0f 1e fa      <endbr64
178 40120c: 55              <push    rbp
179 40120d: 48 89 a5        <mov     rbp, rsp
180 401210: 48 89 7d c8      <mov     QWORD PTR [rbp-0x38], rdi
181 401214: c7 45 fc 00 00 00 00 <mov     DWORD PTR [rbp-0x4], 0x0
182 40121b: eb 04          <jmp     401221 <verificar_licencia+0x19>
183 40121d: 83 45 fc 01      <add     DWORD PTR [rbp-0x4], 0x1
184 401221: 8b 45 fc        <mov     eax, DWORD PTR [rbp-0x4]
185 401224: 48 63 d0        <movsxd  rdx, eax
186 401227: 48 8b 45 c8      <mov     rax, QWORD PTR [rbp-0x38]
187 40122b: 48 01 d0        <add     rax, rdx
188 40122e: 0f b6 00        <movzx   eax, BYTE PTR [rax]
189 401231: 84 c0          <test    al, al
190 401233: 75 e8          <jne     40121d <verificar_licencia+0x15>
191 401235: 83 7d fc 08      <cmp     DWORD PTR [rbp-0x4], 0x8
192 401239: 74 0a          <jle     401245 <verificar_licencia+0x3d>
193 40123b: b8 00 00 00 00 <mov     eax, 0x0
194 401240: a2 20 02 00 00 <jmp     401465 <verificar_licencia+0x25d>
195 401245: 48 8b 45 c8      <mov     rax, QWORD PTR [rbp-0x38]
196 401249: 0f b6 00        <movzx   eax, BYTE PTR [rax]
197 40124c: 3c 40          <cmp     al, 0x40
198 40124e: 7e 0b          <jle     40125b <verificar_licencia+0x53>
199 401250: 48 8b 45 c8      <mov     rax, QWORD PTR [rbp-0x38]
200 401254: 0f b6 00        <movzx   eax, BYTE PTR [rax]
201 401257: 3c 5a          <cmp     al, 0x5a
202 401259: 7e 0a          <jle     401265 <verificar_licencia+0x5d>
203 40125b: b8 00 00 00 00 <mov     eax, 0x0
204 401260: a2 00 02 00 00 <jmp     401465 <verificar_licencia+0x25d>
205 401265: 48 8b 45 c8      <mov     rax, QWORD PTR [rbp-0x38]
206 401269: 48 83 c0 01      <add     rax, 0x1
207 40126d: 0f b6 00        <movzx   eax, BYTE PTR [rax]
208 401270: 3c 40          <cmp     al, 0x40
209 401272: 7e 0f          <jle     401283 <verificar_licencia+0x7b>
210 401274: 48 8b 45 c8      <mov     rax, QWORD PTR [rbp-0x38]
211 401278: 48 83 c0 01      <add     rax, 0x1
212 40127c: 0f b6 00        <movzx   eax, BYTE PTR [rax]
213 40127f: 3c 5a          <cmp     al, 0x5a
214 401281: 7e 0a          <jle     40128d <verificar_licencia+0x85>
215 401283: b8 00 00 00 00 <mov     eax, 0x0
216 401288: a2 d8 01 00 00 <jmp     401465 <verificar_licencia+0x25d>
217 40128d: 48 8b 45 c8      <mov     rax, QWORD PTR [rbp-0x38]
218 401291: 48 83 c0 02      <add     rax, 0x2
219 401295: 0f b6 00        <movzx   eax, BYTE PTR [rax]

```

Esa es la función de verificación identificada, arriba dice el nombre “verificar-licencia”. Es una función de gran tamaño y se observan diferentes comparaciones.

## Fase 2: Análisis y reconstrucción del algoritmo.

(4) Analice las instrucciones de comparación (cmp) y salto condicional (jle, jge, jne, etc.) dentro de la función de validación. ¿Qué rangos de valores ASCII está verificando el programa para cada posición del código de licencia? ¿Qué tipo de caracteres acepta en cada parte?

```

0000000000401208 <verificar_licencia>:
401208:>f3 0f 1e fa      ><endbr64
40120c:>55              ><push    rbp
40120d:>48 89 a5        ><mov     rbp, rsp
401210:>48 89 7d c8      ><mov     QWORD PTR [rbp-0x38], rdi
401214:>c7 45 fc 00 00 00 00 ><mov     DWORD PTR [rbp-0x4], 0x0
40121b:>eb 04          ><jmp     401221 <verificar_licencia+0x19>
40121d:>83 45 fc 01      ><add     DWORD PTR [rbp-0x4], 0x1
401221:>8b 45 fc        ><mov     eax, DWORD PTR [rbp-0x4]
401224:>48 63 d0        ><movsxd  rdx, eax
401227:>48 8b 45 c8      ><mov     rax, QWORD PTR [rbp-0x38]
40122b:>48 01 d0        ><add     rax, rdx
40122e:>0f b6 00        ><movzx   eax, BYTE PTR [rax]
401231:>84 c0          ><test    al, al
401233:>75 e8          ><jne     40121d <verificar_licencia+0x15>
401235:>83 7d fc 08      ><cmp     DWORD PTR [rbp-0x4], 0x8
401239:>74 0a          ><jle     401245 <verificar_licencia+0x3d>
40123b:>b8 00 00 00 00 ><mov     eax, 0x0
401240:>a2 20 02 00 00 ><jmp     401465 <verificar_licencia+0x25d>

```

El primer parámetro de la función está en [rbp-0x38].

De la línea 182 a 190 parece haber un ciclo y en [rbp-4] para ser una variable que denota un contador, el cual se utiliza para verificar que sea de tamaño 8.

```
195 401245: 48 8b 45 c8      mov     rax,QWORD PTR [rbp-0x38]
196 401249: 0f b6 00      movzx   eax,BYTE PTR [rax]
197 40124c: 3c 40      cmp     al,0x40
198 40124e: 7e 0b      jle     40125b <verificar_licencia+0x53>
199 401250: 48 8b 45 c8      mov     rax,QWORD PTR [rbp-0x38]
200 401254: 0f b6 00      movzx   eax,BYTE PTR [rax]
201 401257: 3c 5a      cmp     al,0x5a
202 401259: 7e 0a      jle     401265 <verificar_licencia+0x5d>
```

La primera posición tiene que ser  $\geq$  'A' y  $\leq$  'z'.

La segunda, tercera, cuarta posición también.

```
4012dd: 48 8b 45 c8      mov     rax,QWORD PTR [rbp-0x38]
4012e1: 48 83 c0 04      add     rax,0x4
4012e5: 0f b6 00      movzx   eax,BYTE PTR [rax]
4012e8: 3c 2f      cmp     al,0x2f
4012ea: 7e 0f      jle     4012fb <verificar_licencia+0xf3>
4012ec: 48 8b 45 c8      mov     rax,QWORD PTR [rbp-0x38]
4012f0: 48 83 c0 04      add     rax,0x4
4012f4: 0f b6 00      movzx   eax,BYTE PTR [rax]
4012f7: 3c 39      cmp     al,0x39
4012f9: 7e 0a      jle     401305 <verificar_licencia+0xfd>
4012fb: b8 00 00 00 00  mov     eax,0x0
401300: e9 60 01 00 00  jmp     401465 <verificar_licencia+0x25d>
```

La quinta posición tiene que ser un número del 0 al 9. La sexta, séptima y octava posición también.

En resumen:  $[A-Z]^4[0-9]^4$ .

(5) Identifique las operaciones aritméticas (add, imul, idiv, sub, etc.) presentes en la función de validación. Describa qué operación matemática realiza cada una y sobre qué valores opera. Preste especial atención a las constantes numéricas que aparecen como operandos inmediatos.

```

40137d: 48 8b 45 c8      >mov     rax,QWORD PTR [rbp-0x38]
401381: 0f b6 00         >movzx   eax,BYTE PTR [rax]
401384: 0f be d0         >movsx   edx,al
401387: 48 8b 45 c8      >mov     rax,QWORD PTR [rbp-0x38]
40138b: 48 83 c0 01      >add     rax,0x1
40138f: 0f b6 00         >movzx   eax,BYTE PTR [rax]
401392: 0f be c0         >movsx   eax,al
401395: 01 c2           >add     edx,eax
401397: 48 8b 45 c8      >mov     rax,QWORD PTR [rbp-0x38]
40139b: 48 83 c0 02      >add     rax,0x2
40139f: 0f b6 00         >movzx   eax,BYTE PTR [rax]
4013a2: 0f be c0         >movsx   eax,al
4013a5: 01 c2           >add     edx,eax
4013a7: 48 8b 45 c8      >mov     rax,QWORD PTR [rbp-0x38]
4013ab: 48 83 c0 03      >add     rax,0x3
4013af: 0f b6 00         >movzx   eax,BYTE PTR [rax]
4013b2: 0f be c0         >movsx   eax,al
4013b5: 01 d0           >add     eax,edx
4013b7: 89 45 f8         >mov     DWORD PTR [rbp-0x8],eax

```

En esta parte suma los caracteres de las cuatro primeras posiciones y guarda este valor.

```

306 4013b5: 01 d0         >add     eax,edx
307 4013b7: 89 45 f8         >mov     DWORD PTR [rbp-0x8],eax
308 4013ba: 8b 05 74 2c 00 00 >mov     eax,DWORD PTR [rip+0x2c74]      # 404034 <MULTIPLICADOR>
309 4013c0: 8b 55 f8         >mov     edx,DWORD PTR [rbp-0x8]
310 4013c3: 0f af c2         >imul    eax,edx
311 4013c6: 89 45 f4         >mov     DWORD PTR [rbp-0xc],eax
312 4013c9: 8b 0d 61 2c 00 00 >mov     ecx,DWORD PTR [rip+0x2c61]      # 404030 <MODULO_BASE>
313 4013cf: 8b 45 f4         >mov     eax,DWORD PTR [rbp-0xc]
314 4013d2: 99              >cdq
315 4013d3: f7 f9           >idiv    ecx
316 4013d5: 89 55 f0         >mov     DWORD PTR [rbp-0x10],edx

```

La suma la multiplica por un valor (puede ser una constante) y le saca módulo un valor (constante) y guarda el resultado.

**(6)** A partir de su análisis de las preguntas 4 y 5, escriba el pseudocódigo completo del algoritmo de validación de licencias. El pseudocódigo debe ser lo suficientemente detallado para que alguien que no haya visto el assembly pueda implementar la misma lógica en cualquier lenguaje de programación.

Pseudo código incompleto:

verificar(s):

```

    if size(s) != 8: return false
    val = 0
    for i in [0, 3]:
        val=val+s[i]
    val = (val * MULTIPLICADOR)%MODULO_BASE

```

**(7)** Si utilizó Ghidra u otra herramienta de decompilación, incluya el pseudo-C generado y compárelo con su pseudocódigo manual. ¿Coinciden? ¿En qué difieren? Si no utilizó Ghidra, explique cómo

la herramienta de decompilación podría complementar el análisis manual que realizó.

MULTIPLICADOR=9

MODULO\_BASE=10000

undefined8 verificar\_licencia(char \*param\_1)

```
{
    undefined8 uVar1;
    int local_c;

    for (local_c = 0; param_1[local_c] != '\0'; local_c = local_c + 1) {
    }
    if (local_c == 8) {
        if ((*param_1 < 'A') || ('Z' < *param_1)) {
            uVar1 = 0;
        }
        else if ((param_1[1] < 'A') || ('Z' < param_1[1])) {
            uVar1 = 0;
        }
        else if ((param_1[2] < 'A') || ('Z' < param_1[2])) {
            uVar1 = 0;
        }
        else if ((param_1[3] < 'A') || ('Z' < param_1[3])) {
            uVar1 = 0;
        }
        else if ((param_1[4] < '0') || ('9' < param_1[4])) {
            uVar1 = 0;
        }
        else if ((param_1[5] < '0') || ('9' < param_1[5])) {
            uVar1 = 0;
        }
        else if ((param_1[6] < '0') || ('9' < param_1[6])) {
            uVar1 = 0;
        }
        else if ((param_1[7] < '0') || ('9' < param_1[7])) {
            uVar1 = 0;
        }
        else if (param_1[7] + -0x30 +
            (param_1[4] + -0x30) * 1000 + (param_1[5] + -0x30) * 100 + (param_1[6] + -0x30) *
10 ==
            (MULTIPLICADOR * ((int)param_1[3] + (int)*param_1 + (int)param_1[1] +
(int)param_1[2]))
            % MODULO_BASE) {
            uVar1 = 1;
        }
        else {
            uVar1 = 0;
        }
    }
```

```

    }
}
else {
    uVar1 = 0;
}
return uVar1;
}

```

La primera parte de verificación de rangos coincide con el pseudocódigo que escribí, al igual que la segunda parte de la comparación del último if. Sin embargo, no me dio tiempo de analizar y completar todo el pseudocódigo.

### **Fase 3: Explotación y análisis de debilidades.**

(8) Utilizando el algoritmo reconstruido, genere al menos dos códigos de licencia que el programa acepte como válidos. Para cada código, muestre el trabajo matemático completo paso a paso que demuestre por qué el programa lo aceptaría.

1. AAAA2340 :  $[A-Z]^4 [0-9]^4$

$A=65$

$A+A+A+A=260$

$260*9 \% 10000 = 2340$

$2*1000+3*100*4*10+0 = 2340$

Por lo cual es un código correcto.

2. AAAB2349:  $[A-Z]^4 [0-9]^4$

$A=65$

$B = 66$

$A+A+A+B=65*3+66 =261$

$261*9 \% 10000 = 2349$

$2*1000+3*100*4*10+9 = 2349$

Por lo cual es un código correcto.

(9) Ejecute el programa con los códigos generados y demuestre con capturas de pantalla que son aceptados como válidos. La captura debe mostrar el flag que el programa despliega al aceptar la licencia.

Código 1:

```

.../Music/tcl > ls
Permissions Size User Date Modified Name
-rw-r--r-- 22k kencast 15 May 09:16 desensamblado.txt
-rwxr--r-- 16k kencast 13 May 14:16 software_premium_04
-rw-r--r-- 572k kencast 13 May 15:45 TC1-seguridad.pdf

.../Music/tcl > ./software_premium_04
=====
SOFTWARE PREMIUM v3.2.4
Ingrese su codigo de licencia
=====
Codigo (8 caracteres): AAAA2340
[+] Licencia valida! Funciones premium activadas.
[*] FLAG{SP04_f0e5a2d0}

.../Music/tcl > █

```

Código 2:

```

.../Music/tcl > ./software_premium_04
=====
SOFTWARE PREMIUM v3.2.4
Ingrese su codigo de licencia
=====
Codigo (8 caracteres): AAAB2349
[+] Licencia valida! Funciones premium activadas.
[*] FLAG{SP04_f0e5a2d0}

.../Music/tcl > █

```